Redes GRU (Gated Recurrent Unit)

Contents

- Introducción
- Funcionamiento de una red GRU
- Puerta de actualización-update
- Puerta reinicio
- Activación candidata
- Actualización del estado recurrente
- Resumen de la matemática en la red GRU
- Computación en una capa GRU de Keras
- Referencias

Introducción

Una red neuronal recurrente (RNR) es una extensión de una red neuronal convencional, que es capaz de manejar una entrada de secuencia de longitud variable. La RNR maneja la longitud variable de las secuencias mediante un estado oculto recurrente cuya activación en cada momento depende del estado anterior.

Más formalmente, dada una secuencia $\mathbf{x} = (x_1, x_2, \ldots, x_T)$, la RNR actualiza su estado oculto recurrente h_t mediante

$$h_t = \begin{cases} 0, & \text{si } t = 0, \\ \phi(h_{t-1}, x_t), & \text{en otro caso.} \end{cases}$$

Si g es una función de activación suave, como un sigmoide o una tangente hiperbólica, es común definir

$$h_t = g(Wh_{t-1} + Ux_t + b)$$

Una RNR generativa genera una distribución de probabilidad sobre el siguiente elemento de la secuencia, dado su estado actual h_t, y este modelo generativo puede capturar una distribución sobre secuencias de longitud variable mediante el uso de un símbolo de salida especial para representar el final de la secuencia. La secuencia la probabilidad se puede descomponer en

$$p(x_1, \ldots, x_T) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \ldots p(x_T|x_1, \ldots, x_{T-1}),$$

donde el último elemento es un valor especial de final de secuencia. Modelamos cada probabilidad condicional distribución con

$$p(x_t|x_1, \ldots, x_{t-1}) = g(h_t)$$

El problema con este modelo, es que el cálculo del gradiente tiende a volverse cero o a explotar.

Dos líneas de trabajo impulsó esta situación. Por un lado se inició la búsqueda de nuevas técnicas para para el uso del gradiente en el proceso de optimización de la función de costo y por el otro, el desarrollo de nuevos modelos de redes neuronales.

La primera línea ha producido nuevas técnicas de optimización estocástica basados en el gradiente, que han sido usado exitosamente en redes generales.

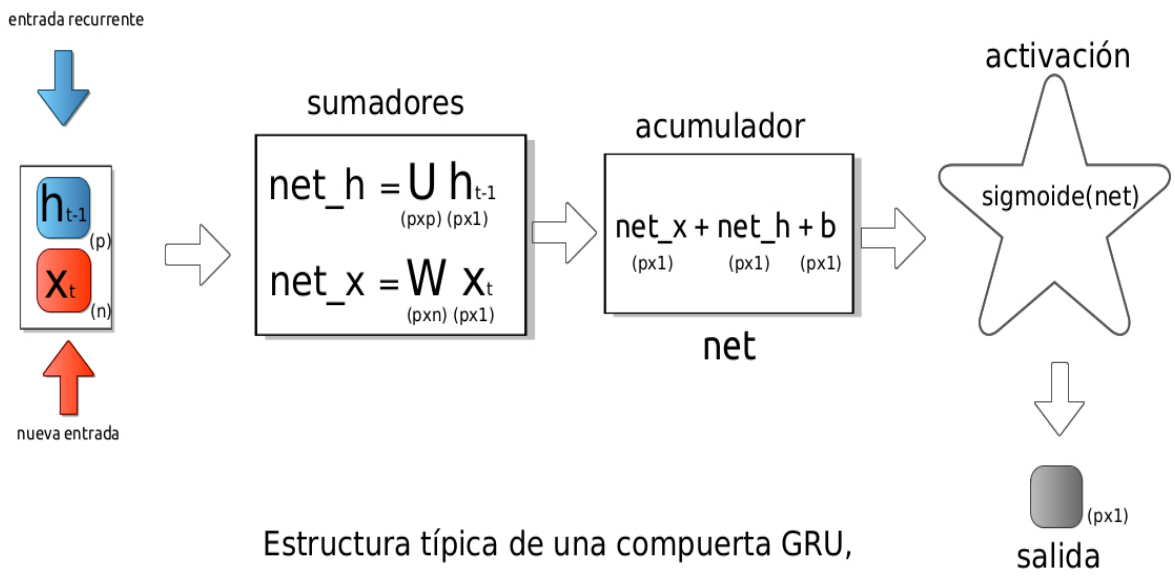
La segunda línea llevó al desarrollo de las redes LSTM, en las cuales la función de activación consiste en una transformación afinada seguida por una simple no linealidad de elementos mediante el uso de unidades de compuerta, Hochreiter y Schmidhuber, 1997.

Más recientemente, otro tipo de unidad recurrente, a la que nos referimos como una unidad recurrente cerrada (GRU), fue propuesta por Cho et al. 2014. Puede consultar una comparación entre LSTM y GRU en Junyoung Chung et al., 2014. De estas unidades recurrentes se ha demostrado que funcionan bien en tareas que requieren captura de dependencias a largo plazo. Esas tareas incluyen, pero no se limitan a reconocimiento de voz, música,...

Funcionamiento de una red GRU

Una red de unidad recurrente cerrada (GRU) permite que cada unidad recurrente capture de forma adaptativa dependencias de diferentes escalas de tiempo. De manera similar a la unidad LSTM, la GRU tiene puertas que modulan el flujo de información dentro de la unidad. Sin embargo, a diferencia de las redes LSTM no tiene celdas de memoria separadas. Las dos imágenes muestran la estructura general de las compuertas GRU.

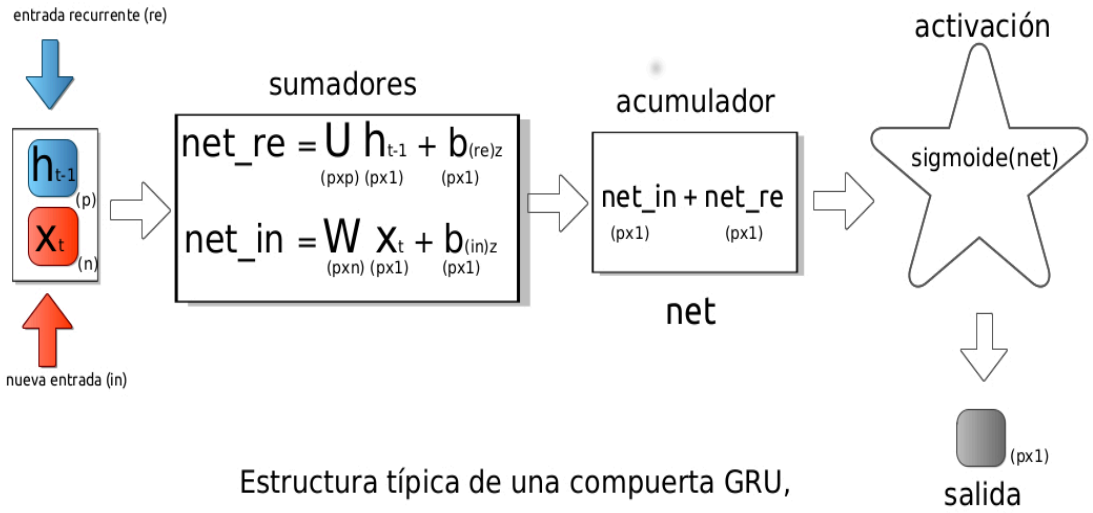
Estos diagramas muestran la estructura matemática dentro de la compuerta en el tiempo t, con reinicio antes o después de de la multiplicación.



Estructura típica de una compuerta GRU, con reinicio después de multiplicación (reset_after=False)

Imagen: Alvaro Montenegro

reset_after: convención GRU (si aplicar la puerta de reinicio después o antes de la multiplicación de matrices). False = «antes», True = «después» (este es el caso por defecto).



Estructura típica de una compuerta GRU, con reinicio antes de multiplicación (reset_after=True)

Imagen: Alvaro Montenegro

La notación que usamos es bastante estándar.

1. $+$: indica suma de vectores
2. σ : representa a la función de activación sigmoide
3. $\tanh$: representa a la función de activación tangente hiperbólica. 4.$\odot$: es producto componente a componente (producto de Hamard)

Una GRU tien dos tipos de puerta: actualización (update) y reinicio (reset).

Puerta de actualización-update

Comenzamos calculando la puerta de actualización z_t para el paso de tiempo t usando la fórmula:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

en donde W_z y U_z son pesos asociados x_t (la nueva entrada) y h_{t-1}, (la información procedente de la unidad anterior).

La puerta de actualización ayuda al modelo a determinar qué cantidad de la información pasada (de los pasos de tiempo anteriores) debe transmitirse al futuro, combinandola con la nueva información.

Eso es realmente poderoso porque el modelo puede decidir copiar toda la información del pasado y eliminar el riesgo de desvanecer del gradiente.

Puerta reinicio

Esencialmente, esta puerta se utiliza para que el modelo decida qué cantidad de información pasada debe olvidar. Para calcularla, utilizamos:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r),$$

en donde W_r y U_r son pesos asociados x_t (la nueva entrada) y h_{t-1}, (la información procedente de la unidad anterior).

Activación candidata

Veamos cómo afectarán exactamente las puertas al resultado final. Primero, comenzamos con el uso de la puerta de reinicio. Introducimos un nuevo contenido de memoria que utilizará la puerta de reinicio para almacenar la información relevante del pasado. Se calcula de la siguiente manera:

$$\tilde{h}_t = \tanh(W_n x_t + U_n(r_t \odot h_{t-1}) + b_n),$$

en donde W_n y U_n son pesos asociados a las x's y a las h's respectivamente.

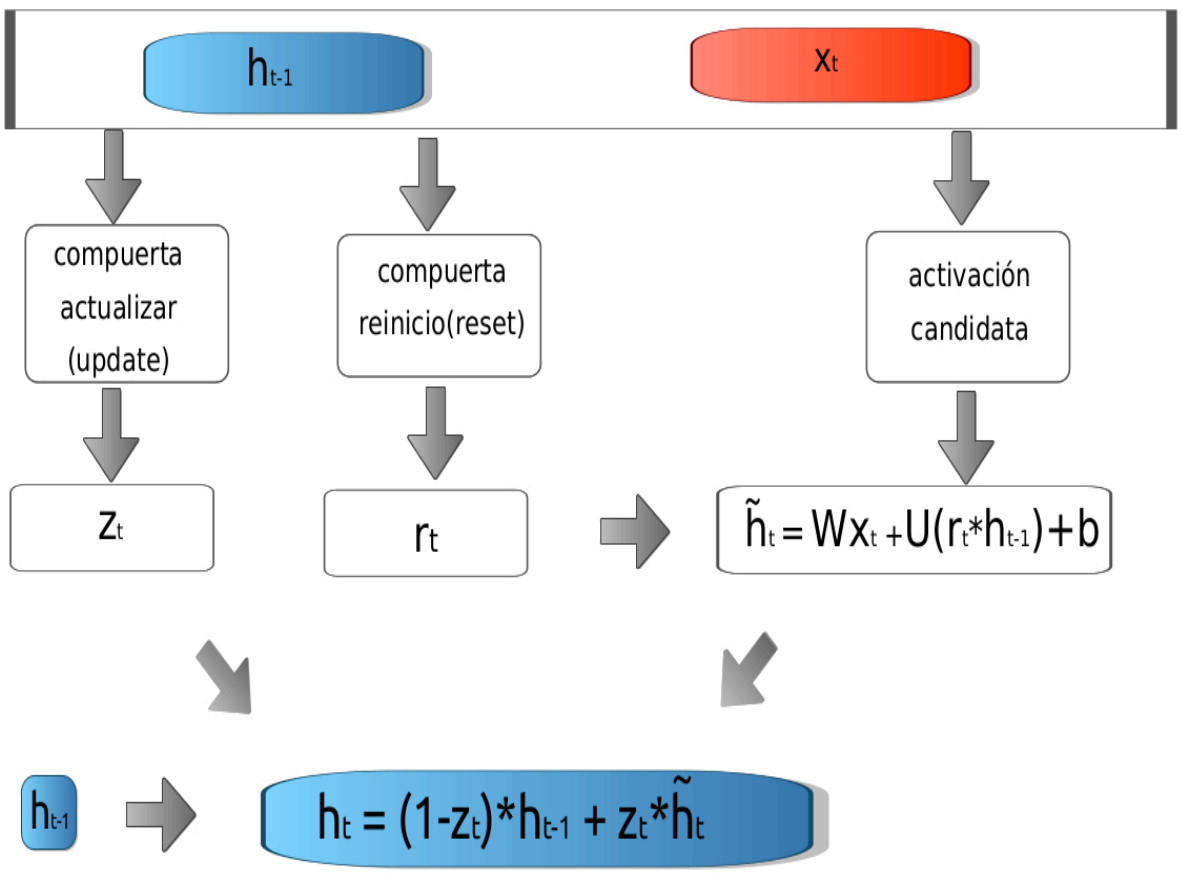
Actualización del estado recurrente

La activación de la GRU en tiempo t es una interpolación lineal entre la activación previa h_{t-1} y la activación candidata $\tilde{h}_t$ definida arriba.

En símbolos tenemos:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

La siguiente imagen ilustra la arquitectura de la red.



Arquitectura de una red GRU

Imagen: Alvaro Montenegro

Resumen de la matemática en la red GRU

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r),$$

$$\tilde{h}_t = \tanh(W_n x_t + U_n(r_t \odot h_{t-1}) + b_n),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Computación en una capa GRU de Keras

Cualquier capa de Keras siempre espera un batch de datos. En el caso de una capa LSTM Keras espera tensores 3D de la siguiente forma

- [batch_size, time_step, feature]

Por ejemplo un tensor de entrada de tamaño [32, 10, 8], la capa Keras lo interpreta como

- batch_size = 32, es decir 32 ejemplos.
- time_step = 10, es decir secuencias de entrada de tamaño 10. Por ejemplo en una serie de tiempo este es el tamaño de la ventana de entrada.
- feature = 8, es decir la variable de entrada es de tamaño 8. Por ejemplo, en series de timepo univariadas, feature = 1. En una serie multivariada con 8 variables, features = 8. En modelos de lenguaje natural feature = tamaño de representación de acada token. Usualmente correspondería al tamaño del embedding.

La salida de la capa corresponde al tamaño del estado oculto. Por ejemplo, si el estado oculto tiene tamaño, la salida de la capa es de tamaño [batch_size, 4].

Ejemplo

```
import tensorflow as tf
inputs = tf.random.normal([32,10,8])
gru = tf.keras.layers.GRU(4) # gru es una capa de tamaño de salidad 4.
output = gru(inputs)
print (output.shape)
```

```
(32, 4)
```

Recibiendo toda la secuencia del valor del estado oculto

En algunos casos es necesario disponer del valor del estado oculto para cada valor en la secuenci de entrada. Esta secuencia tiene tamaño [batch_size, time_step, output_size] En el sugiente ejemplo se tiene que

- `return_sequences` son todos lo estados del estado oculto
- `return_state` es el último valor del estado oculto

```
gru = tf.keras.layers.GRU(4, return_sequences = True, return_state=True)
whole_seq_output, final_memory_state = gru(inputs)
```

```
print(whole_seq_output.shape)
print(final_memory_state.shape)
```

```
(32, 10, 4)
(32, 4)
```

```
whole_seq_output[:,-1] == final_memory_state
```

```
<tf.Tensor: shape=(32, 4), dtype=bool, numpy=
array([[ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True]])>
```

Extracción de los pesos de la capa

```
help(gru)
```

```
Help on GRU in module keras.layers.rnn.gru object:

class GRU(keras.layers.rnn.dropout_rnn_cell_mixin.DropoutRNNCellMixin,
keras.layers.rnn.base_rnn.RNN, keras.engine.base_layer.BaseRandomLayer)
 |  GRU(units, activation='tanh', recurrent_activation='sigmoid', use_bias=True,
kernel_initializer='glorot_uniform', recurrent_initializer='orthogonal',
bias_initializer='zeros', kernel_regularizer=None, recurrent_regularizer=None,
bias_regularizer=None, activity_regularizer=None, kernel_constraint=None,
recurrent_constraint=None, bias_constraint=None, dropout=0.0, recurrent_dropout=0.0,
return_sequences=False, return_state=False, go_backwards=False, stateful=False,
unroll=False, time_major=False, reset_after=True, **kwargs)
 |
 |  Gated Recurrent Unit - Cho et al. 2014.
 |
 |  See [the Keras RNN API guide](https://www.tensorflow.org/guide/keras/rnn)
 |  for details about the usage of RNN API.
 |
 |  Based on available runtime hardware and constraints, this layer
 |  will choose different implementations (cuDNN-based or pure-TensorFlow)
 |  to maximize the performance. If a GPU is available and all
 |  the arguments to the layer meet the requirement of the cuDNN kernel
 |  (see below for details), the layer will use a fast cuDNN implementation.
 |
 |  The requirements to use the cuDNN implementation are:
 |
 |  1. `activation` == `tanh`
 |  2. `recurrent_activation` == `sigmoid`
 |  3. `recurrent_dropout` == 0
 |  4. `unroll` is `False`
 |  5. `use_bias` is `True`
 |  6. `reset_after` is `True`
 |  7. Inputs, if use masking, are strictly right-padded.
 |  8. Eager execution is enabled in the outermost context.
 |
 |  There are two variants of the GRU implementation. The default one is based on
 |  [v3](https://arxiv.org/abs/1406.1078v3) and has reset gate applied to hidden
 |  state before matrix multiplication. The other one is based on
 |  [original](https://arxiv.org/abs/1406.1078v1) and has the order reversed.
 |
 |  The second variant is compatible with CuDNNGRU (GPU-only) and allows
 |  inference on CPU. Thus it has separate biases for `kernel` and
 |  `recurrent_kernel`. To use this variant, set `reset_after=True` and
 |  `recurrent_activation='sigmoid'`.
 |
 |  For example:
 |
 |  >>> inputs = tf.random.normal([32, 10, 8])
 |  >>> gru = tf.keras.layers.GRU(4)
 |  >>> output = gru(inputs)
 |  >>> print(output.shape)
 |  (32, 4)
 |  >>> gru = tf.keras.layers.GRU(4, return_sequences=True, return_state=True)
 |  >>> whole_sequence_output, final_state = gru(inputs)
 |  >>> print(whole_sequence_output.shape)
 |  (32, 10, 4)
 |  >>> print(final_state.shape)
 |  (32, 4)
 |
 |  Args:
 |    units: Positive integer, dimensionality of the output space.
 |    activation: Activation function to use.
 |      Default: hyperbolic tangent (`tanh`).
 |      If you pass `None`, no activation is applied
 |      (ie. "linear" activation: `a(x) = x`).
 |    recurrent_activation: Activation function to use
 |      for the recurrent step.
 |      Default: sigmoid (`sigmoid`).
 |      If you pass `None`, no activation is applied
 |      (ie. "linear" activation: `a(x) = x`).
 |    use_bias: Boolean, (default `True`), whether the layer uses a bias vector.
 |    kernel_initializer: Initializer for the `kernel` weights matrix,
 |      used for the linear transformation of the inputs. Default:
 |      `glorot_uniform`.
 |    recurrent_initializer: Initializer for the `recurrent_kernel`
 |       weights matrix, used for the linear transformation of the recurrent
 |       state. Default: `orthogonal`.
 |    bias_initializer: Initializer for the bias vector. Default: `zeros`.
 |    kernel_regularizer: Regularizer function applied to the `kernel` weights
 |      matrix. Default: `None`.
 |    recurrent_regularizer: Regularizer function applied to the
 |      `recurrent_kernel` weights matrix. Default: `None`.
 |    bias_regularizer: Regularizer function applied to the bias vector. Default:
 |      `None`.
 |    activity_regularizer: Regularizer function applied to the output of the
 |      layer (its "activation"). Default: `None`.
```

```
|    kernel_constraint: Constraint function applied to the `kernel` weights
|      matrix. Default: `None`.
|    recurrent_constraint: Constraint function applied to the `recurrent_kernel`
|      weights matrix. Default: `None`.
|    bias_constraint: Constraint function applied to the bias vector. Default:
|      `None`.
|    dropout: Float between 0 and 1. Fraction of the units to drop for the linear
|      transformation of the inputs. Default: 0.
|    recurrent_dropout: Float between 0 and 1. Fraction of the units to drop for
|      the linear transformation of the recurrent state. Default: 0.
|    return_sequences: Boolean. Whether to return the last output
|      in the output sequence, or the full sequence. Default: `False`.
|    return_state: Boolean. Whether to return the last state in addition to the
|      output. Default: `False`.
|    go_backwards: Boolean (default `False`).
|      If True, process the input sequence backwards and return the
|      reversed sequence.
|    stateful: Boolean (default False). If True, the last state
|      for each sample at index i in a batch will be used as initial
|      state for the sample of index i in the following batch.
|    unroll: Boolean (default False).
|      If True, the network will be unrolled,
|      else a symbolic loop will be used.
|      Unrolling can speed-up a RNN,
|      although it tends to be more memory-intensive.
|      Unrolling is only suitable for short sequences.
|    time_major: The shape format of the `inputs` and `outputs` tensors.
|      If True, the inputs and outputs will be in shape
|      `[timesteps, batch, feature]`, whereas in the False case, it will be
|      `[batch, timesteps, feature]`. Using `time_major = True` is a bit more
|      efficient because it avoids transposes at the beginning and end of the
|      RNN calculation. However, most TensorFlow data is batch-major, so by
|      default this function accepts input and emits output in batch-major
|      form.
|    reset_after: GRU convention (whether to apply reset gate after or
|      before matrix multiplication). False = "before",
|      True = "after" (default and cuDNN compatible).
|
|  Call arguments:
|    inputs: A 3D tensor, with shape `[batch, timesteps, feature]`.
|    mask: Binary tensor of shape `[samples, timesteps]` indicating whether
|      a given timestep should be masked  (optional, defaults to `None`).
|      An individual `True` entry indicates that the corresponding timestep
|      should be utilized, while a `False` entry indicates that the
|      corresponding timestep should be ignored.
|    training: Python boolean indicating whether the layer should behave in
|      training mode or in inference mode. This argument is passed to the cell
|      when calling it. This is only relevant if `dropout` or
|      `recurrent_dropout` is used  (optional, defaults to `None`).
|    initial_state: List of initial state tensors to be passed to the first
|      call of the cell  (optional, defaults to `None` which causes creation
|      of zero-filled initial state tensors).
|
|  Method resolution order:
|      GRU
|      keras.layers.rnn.dropout_rnn_cell_mixin.DropoutRNNCellMixin
|      keras.layers.rnn.base_rnn.RNN
|      keras.engine.base_layer.BaseRandomLayer
|      keras.engine.base_layer.Layer
|      tensorflow.python.module.module.Module
|      tensorflow.python.training.tracking.autotrackable.AutoTrackable
|      tensorflow.python.training.tracking.base.Trackable
|      keras.utils.version_utils.LayerVersionSelector
|      builtins.object
|
|  Methods defined here:
|
|  __init__(self, units, activation='tanh', recurrent_activation='sigmoid',
use_bias=True, kernel_initializer='glorot_uniform',
recurrent_initializer='orthogonal', bias_initializer='zeros',
kernel_regularizer=None, recurrent_regularizer=None, bias_regularizer=None,
activity_regularizer=None, kernel_constraint=None, recurrent_constraint=None,
bias_constraint=None, dropout=0.0, recurrent_dropout=0.0, return_sequences=False,
return_state=False, go_backwards=False, stateful=False, unroll=False,
time_major=False, reset_after=True, **kwargs)
|      Initialize the BaseRandomLayer.
|
|      Note that the constructor is annotated with
|      @no_automatic_dependency_tracking. This is to skip the auto
|      tracking of self._random_generator instance, which is an AutoTrackable.
|      The backend.RandomGenerator could contain a tf.random.Generator instance
|      which will have tf.Variable as the internal state. We want to avoid saving
|      that state into model.weights and checkpoints for backward compatibility
|      reason. In the meantime, we still need to make them visible to SavedModel
|      when it is tracing the tf.function for the `call()`.
|      See _list_extra_dependencies_for_serialization below for more details.
```

```
|
|      Args:
|        seed: optional integer, used to create RandomGenerator.
|        force_generator: boolean, default to False, whether to force the
|          RandomGenerator to use the code branch of tf.random.Generator.
|        **kwargs: other keyword arguments that will be passed to the parent class
|
|  call(self, inputs, mask=None, training=None, initial_state=None)
|      This is where the layer's logic lives.
|
|      The `call()` method may not create state (except in its first invocation,
|      wrapping the creation of variables or other resources in `tf.init_scope()`).
|      It is recommended to create state in `__init__()`, or the `build()` method
|      that is called automatically before `call()` executes the first time.
|
|      Args:
|        inputs: Input tensor, or dict/list/tuple of input tensors.
|          The first positional `inputs` argument is subject to special rules:
|          - `inputs` must be explicitly passed. A layer cannot have zero
|            arguments, and `inputs` cannot be provided via the default value
|            of a keyword argument.
|          - NumPy array or Python scalar values in `inputs` get cast as tensors.
|          - Keras mask metadata is only collected from `inputs`.
|          - Layers are built (`build(input_shape)` method)
|            using shape info from `inputs` only.
|          - `input_spec` compatibility is only checked against `inputs`.
|          - Mixed precision input casting is only applied to `inputs`.
|            If a layer has tensor arguments in `*args` or `**kwargs`, their
|            casting behavior in mixed precision should be handled manually.
|          - The SavedModel input specification is generated using `inputs` only.
|          - Integration with various ecosystem packages like TFMOT, TFLite,
|            TF.js, etc is only supported for `inputs` and not for tensors in
|            positional and keyword arguments.
|        *args: Additional positional arguments. May contain tensors, although
|          this is not recommended, for the reasons above.
|        **kwargs: Additional keyword arguments. May contain tensors, although
|          this is not recommended, for the reasons above.
|          The following optional keyword arguments are reserved:
|          - `training`: Boolean scalar tensor of Python boolean indicating
|            whether the `call` is meant for training or inference.
|          - `mask`: Boolean input mask. If the layer's `call()` method takes a
|            `mask` argument, its default value will be set to the mask generated
|            for `inputs` by the previous layer (if `input` did come from a layer
|            that generated a corresponding mask, i.e. if it came from a Keras
|            layer with masking support).
|
|      Returns:
|        A tensor or list/tuple of tensors.
|
|  get_config(self)
|      Returns the config of the layer.
|
|      A layer config is a Python dictionary (serializable)
|      containing the configuration of a layer.
|      The same layer can be reinstantiated later
|      (without its trained weights) from this configuration.
|
|      The config of a layer does not include connectivity
|      information, nor the layer class name. These are handled
|      by `Network` (one layer of abstraction above).
|
|      Note that `get_config()` does not guarantee to return a fresh copy of dict
|      every time it is called. The callers should make a copy of the returned dict
|      if they want to modify it.
|
|      Returns:
|          Python dictionary.
|
|  ----------------------------------------------------------------------
|  Class methods defined here:
|
|  from_config(config) from builtins.type
|      Creates a layer from its config.
|
|      This method is the reverse of `get_config`,
|      capable of instantiating the same layer from the config
|      dictionary. It does not handle layer connectivity
|      (handled by Network), nor weights (handled by `set_weights`).
|
|      Args:
|          config: A Python dictionary, typically the
|              output of get_config.
|
|      Returns:
|          A layer instance.
|
```

```
 |  ----------------------------------------------------------------------
 |  Readonly properties defined here:
 |
 |  activation
 |
 |  bias_constraint
 |
 |  bias_initializer
 |
 |  bias_regularizer
 |
 |  dropout
 |
 |  implementation
 |
 |  kernel_constraint
 |
 |  kernel_initializer
 |
 |  kernel_regularizer
 |
 |  recurrent_activation
 |
 |  recurrent_constraint
 |
 |  recurrent_dropout
 |
 |  recurrent_initializer
 |
 |  recurrent_regularizer
 |
 |  reset_after
 |
 |  units
 |
 |  use_bias
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from
keras.layers.rnn.dropout_rnn_cell_mixin.DropoutRNNCellMixin:
 |
 |  __getstate__(self)
 |
 |  __setstate__(self, state)
 |
 |  get_dropout_mask_for_cell(self, inputs, training, count=1)
 |      Get the dropout mask for RNN cell's input.
 |
 |      It will create mask based on context if there isn't any existing cached
 |      mask. If a new mask is generated, it will update the cache in the cell.
 |
 |      Args:
 |        inputs: The input tensor whose shape will be used to generate dropout
 |          mask.
 |        training: Boolean tensor, whether its in training mode, dropout will be
 |          ignored in non-training mode.
 |        count: Int, how many dropout mask will be generated. It is useful for cell
 |          that has internal weights fused together.
 |      Returns:
 |        List of mask tensor, generated or cached mask based on context.
 |
 |  get_recurrent_dropout_mask_for_cell(self, inputs, training, count=1)
 |      Get the recurrent dropout mask for RNN cell.
 |
 |      It will create mask based on context if there isn't any existing cached
 |      mask. If a new mask is generated, it will update the cache in the cell.
 |
 |      Args:
 |        inputs: The input tensor whose shape will be used to generate dropout
 |          mask.
 |        training: Boolean tensor, whether its in training mode, dropout will be
 |          ignored in non-training mode.
 |        count: Int, how many dropout mask will be generated. It is useful for cell
 |          that has internal weights fused together.
 |      Returns:
 |        List of mask tensor, generated or cached mask based on context.
 |
 |  reset_dropout_mask(self)
 |      Reset the cached dropout masks if any.
 |
 |      This is important for the RNN layer to invoke this in it `call()` method so
 |      that the cached mask is cleared before calling the `cell.call()`. The mask
 |      should be cached across the timestep within the same batch, but shouldn't
 |      be cached between batches. Otherwise it will introduce unreasonable bias
 |      against certain index of data within the batch.
 |
```

```
 |  reset_recurrent_dropout_mask(self)
 |      Reset the cached recurrent dropout masks if any.
 |
 |      This is important for the RNN layer to invoke this in it call() method so
 |      that the cached mask is cleared before calling the cell.call(). The mask
 |      should be cached across the timestep within the same batch, but shouldn't
 |      be cached between batches. Otherwise it will introduce unreasonable bias
 |      against certain index of data within the batch.
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors inherited from
keras.layers.rnn.dropout_rnn_cell_mixin.DropoutRNNCellMixin:
 |
 |  __dict__
 |      dictionary for instance variables (if defined)
 |
 |  __weakref__
 |      list of weak references to the object (if defined)
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from keras.layers.rnn.base_rnn.RNN:
 |
 |  __call__(self, inputs, initial_state=None, constants=None, **kwargs)
 |      Wraps `call`, applying pre- and post-processing steps.
 |
 |      Args:
 |        *args: Positional arguments to be passed to `self.call`.
 |        **kwargs: Keyword arguments to be passed to `self.call`.
 |
 |      Returns:
 |        Output tensor(s).
 |
 |      Note:
 |        - The following optional keyword arguments are reserved for specific uses:
 |          * `training`: Boolean scalar tensor of Python boolean indicating
 |            whether the `call` is meant for training or inference.
 |          * `mask`: Boolean input mask.
 |        - If the layer's `call` method takes a `mask` argument (as some Keras
 |          layers do), its default value will be set to the mask generated
 |          for `inputs` by the previous layer (if `input` did come from
 |          a layer that generated a corresponding mask, i.e. if it came from
 |          a Keras layer with masking support.
 |        - If the layer is not built, the method will call `build`.
 |
 |      Raises:
 |        ValueError: if the layer's `call` method returns None (an invalid value).
 |        RuntimeError: if `super().__init__()` was not called in the constructor.
 |
 |  build(self, input_shape)
 |      Creates the variables of the layer (optional, for subclass implementers).
 |
 |      This is a method that implementers of subclasses of `Layer` or `Model`
 |      can override if they need a state-creation step in-between
 |      layer instantiation and layer call. It is invoked automatically before
 |      the first execution of `call()`.
 |
 |      This is typically used to create the weights of `Layer` subclasses
 |      (at the discretion of the subclass implementer).
 |
 |      Args:
 |        input_shape: Instance of `TensorShape`, or list of instances of
 |          `TensorShape` if the layer expects a list of inputs
 |          (one instance per input).
 |
 |  compute_mask(self, inputs, mask)
 |      Computes an output mask tensor.
 |
 |      Args:
 |          inputs: Tensor or list of tensors.
 |          mask: Tensor or list of tensors.
 |
 |      Returns:
 |          None or a tensor (or list of tensors,
 |              one per output tensor of the layer).
 |
 |  compute_output_shape(self, input_shape)
 |      Computes the output shape of the layer.
 |
 |      This method will cause the layer's state to be built, if that has not
 |      happened before. This requires that the layer will later be used with
 |      inputs that match the input shape provided here.
 |
 |      Args:
 |          input_shape: Shape tuple (tuple of integers)
 |              or list of shape tuples (one per output tensor of the layer).
 |              Shape tuples can include None for free dimensions,
```

```
|              instead of an integer.
|
|      Returns:
|          An input shape tuple.
|
|  get_initial_state(self, inputs)
|
|  reset_states(self, states=None)
|      Reset the recorded states for the stateful RNN layer.
|
|      Can only be used when RNN layer is constructed with `stateful` = `True`.
|      Args:
|        states: Numpy arrays that contains the value for the initial state, which
|          will be feed to cell at the first time step. When the value is None,
|          zero filled numpy array will be created based on the cell state size.
|
|      Raises:
|        AttributeError: When the RNN layer is not stateful.
|        ValueError: When the batch size of the RNN layer is unknown.
|        ValueError: When the input numpy array is not compatible with the RNN
|          layer state, either size wise or dtype wise.
|
|  ----------------------------------------------------------------------
|  Data descriptors inherited from keras.layers.rnn.base_rnn.RNN:
|
|  states
|
|  ----------------------------------------------------------------------
|  Methods inherited from keras.engine.base_layer.Layer:
|
|  __delattr__(self, name)
|      Implement delattr(self, name).
|
|  __setattr__(self, name, value)
|      Support self.foo = trackable syntax.
|
|  add_loss(self, losses, **kwargs)
|      Add loss tensor(s), potentially dependent on layer inputs.
|
|      Some losses (for instance, activity regularization losses) may be dependent
|      on the inputs passed when calling a layer. Hence, when reusing the same
|      layer on different inputs `a` and `b`, some entries in `layer.losses` may
|      be dependent on `a` and some on `b`. This method automatically keeps track
|      of dependencies.
|
|      This method can be used inside a subclassed layer or model's `call`
|      function, in which case `losses` should be a Tensor or list of Tensors.
|
|      Example:
|
|      ```python
|      class MyLayer(tf.keras.layers.Layer):
|        def call(self, inputs):
|          self.add_loss(tf.abs(tf.reduce_mean(inputs)))
|          return inputs
|      ```
|
|      This method can also be called directly on a Functional Model during
|      construction. In this case, any loss Tensors passed to this Model must
|      be symbolic and be able to be traced back to the model's `Input`s. These
|      losses become part of the model's topology and are tracked in `get_config`.
|
|      Example:
|
|      ```python
|      inputs = tf.keras.Input(shape=(10,))
|      x = tf.keras.layers.Dense(10)(inputs)
|      outputs = tf.keras.layers.Dense(1)(x)
|      model = tf.keras.Model(inputs, outputs)
|      # Activity regularization.
|      model.add_loss(tf.abs(tf.reduce_mean(x)))
|      ```
|
|      If this is not the case for your loss (if, for example, your loss references
|      a `Variable` of one of the model's layers), you can wrap your loss in a
|      zero-argument lambda. These losses are not tracked as part of the model's
|      topology since they can't be serialized.
|
|      Example:
|
|      ```python
|      inputs = tf.keras.Input(shape=(10,))
|      d = tf.keras.layers.Dense(10)
|      x = d(inputs)
|      outputs = tf.keras.layers.Dense(1)(x)
|      model = tf.keras.Model(inputs, outputs)
```

```
|      # Weight regularization.
|      model.add_loss(lambda: tf.reduce_mean(d.kernel))
|      ```
|
|      Args:
|        losses: Loss tensor, or list/tuple of tensors. Rather than tensors, losses
|          may also be zero-argument callables which create a loss tensor.
|        **kwargs: Used for backwards compatibility only.
|
|  add_metric(self, value, name=None, **kwargs)
|      Adds metric tensor to the layer.
|
|      This method can be used inside the `call()` method of a subclassed layer
|      or model.
|
|      ```python
|      class MyMetricLayer(tf.keras.layers.Layer):
|        def __init__(self):
|          super(MyMetricLayer, self).__init__(name='my_metric_layer')
|          self.mean = tf.keras.metrics.Mean(name='metric_1')
|
|        def call(self, inputs):
|          self.add_metric(self.mean(inputs))
|          self.add_metric(tf.reduce_sum(inputs), name='metric_2')
|          return inputs
|      ```
|
|      This method can also be called directly on a Functional Model during
|      construction. In this case, any tensor passed to this Model must
|      be symbolic and be able to be traced back to the model's `Input`s. These
|      metrics become part of the model's topology and are tracked when you
|      save the model via `save()`.
|
|      ```python
|      inputs = tf.keras.Input(shape=(10,))
|      x = tf.keras.layers.Dense(10)(inputs)
|      outputs = tf.keras.layers.Dense(1)(x)
|      model = tf.keras.Model(inputs, outputs)
|      model.add_metric(math_ops.reduce_sum(x), name='metric_1')
|      ```
|
|      Note: Calling `add_metric()` with the result of a metric object on a
|      Functional Model, as shown in the example below, is not supported. This is
|      because we cannot trace the metric result tensor back to the model's inputs.
|
|      ```python
|      inputs = tf.keras.Input(shape=(10,))
|      x = tf.keras.layers.Dense(10)(inputs)
|      outputs = tf.keras.layers.Dense(1)(x)
|      model = tf.keras.Model(inputs, outputs)
|      model.add_metric(tf.keras.metrics.Mean()(x), name='metric_1')
|      ```
|
|      Args:
|        value: Metric tensor.
|        name: String metric name.
|        **kwargs: Additional keyword arguments for backward compatibility.
|          Accepted values:
|          `aggregation` - When the `value` tensor provided is not the result of
|          calling a `keras.Metric` instance, it will be aggregated by default
|          using a `keras.Metric.Mean`.
|
|  add_update(self, updates)
|      Add update op(s), potentially dependent on layer inputs.
|
|      Weight updates (for instance, the updates of the moving mean and variance
|      in a BatchNormalization layer) may be dependent on the inputs passed
|      when calling a layer. Hence, when reusing the same layer on
|      different inputs `a` and `b`, some entries in `layer.updates` may be
|      dependent on `a` and some on `b`. This method automatically keeps track
|      of dependencies.
|
|      This call is ignored when eager execution is enabled (in that case, variable
|      updates are run on the fly and thus do not need to be tracked for later
|      execution).
|
|      Args:
|        updates: Update op, or list/tuple of update ops, or zero-arg callable
|          that returns an update op. A zero-arg callable should be passed in
|          order to disable running the updates by setting `trainable=False`
|          on this Layer, when executing in Eager mode.
|
|  add_variable(self, *args, **kwargs)
|      Deprecated, do NOT use! Alias for `add_weight`.
|
|  add_weight(self, name=None, shape=None, dtype=None, initializer=None,
```

```
regularizer=None, trainable=None, constraint=None, use_resource=None,
synchronization=<VariableSynchronization.AUTO: 0>, aggregation=
<VariableAggregationV2.NONE: 0>, **kwargs)
 |      Adds a new variable to the layer.
 |
 |      Args:
 |        name: Variable name.
 |        shape: Variable shape. Defaults to scalar if unspecified.
 |        dtype: The type of the variable. Defaults to `self.dtype`.
 |        initializer: Initializer instance (callable).
 |        regularizer: Regularizer instance (callable).
 |        trainable: Boolean, whether the variable should be part of the layer's
 |          "trainable_variables" (e.g. variables, biases)
 |          or "non_trainable_variables" (e.g. BatchNorm mean and variance).
 |          Note that `trainable` cannot be `True` if `synchronization`
 |          is set to `ON_READ`.
 |        constraint: Constraint instance (callable).
 |        use_resource: Whether to use `ResourceVariable`.
 |        synchronization: Indicates when a distributed a variable will be
 |          aggregated. Accepted values are constants defined in the class
 |          `tf.VariableSynchronization`. By default the synchronization is set to
 |          `AUTO` and the current `DistributionStrategy` chooses
 |          when to synchronize. If `synchronization` is set to `ON_READ`,
 |          `trainable` must not be set to `True`.
 |        aggregation: Indicates how a distributed variable will be aggregated.
 |          Accepted values are constants defined in the class
 |          `tf.VariableAggregation`.
 |        **kwargs: Additional keyword arguments. Accepted values are `getter`,
 |          `collections`, `experimental_autocast` and `caching_device`.
 |
 |      Returns:
 |        The variable created.
 |
 |      Raises:
 |        ValueError: When giving unsupported dtype and no initializer or when
 |          trainable has been set to True with synchronization set as `ON_READ`.
 |
 |  compute_output_signature(self, input_signature)
 |      Compute the output tensor signature of the layer based on the inputs.
 |
 |      Unlike a TensorShape object, a TensorSpec object contains both shape
 |      and dtype information for a tensor. This method allows layers to provide
 |      output dtype information if it is different from the input dtype.
 |      For any layer that doesn't implement this function,
 |      the framework will fall back to use `compute_output_shape`, and will
 |      assume that the output dtype matches the input dtype.
 |
 |      Args:
 |        input_signature: Single TensorSpec or nested structure of TensorSpec
 |          objects, describing a candidate input for the layer.
 |
 |      Returns:
 |        Single TensorSpec or nested structure of TensorSpec objects, describing
 |          how the layer would transform the provided input.
 |
 |      Raises:
 |        TypeError: If input_signature contains a non-TensorSpec object.
 |
 |  count_params(self)
 |      Count the total number of scalars composing the weights.
 |
 |      Returns:
 |          An integer count.
 |
 |      Raises:
 |          ValueError: if the layer isn't yet built
 |            (in which case its weights aren't yet defined).
 |
 |  finalize_state(self)
 |      Finalizes the layers state after updating layer weights.
 |
 |      This function can be subclassed in a layer and will be called after updating
 |      a layer weights. It can be overridden to finalize any additional layer state
 |      after a weight update.
 |
 |      This function will be called after weights of a layer have been restored
 |      from a loaded model.
 |
 |  get_input_at(self, node_index)
 |      Retrieves the input tensor(s) of a layer at a given node.
 |
 |      Args:
 |          node_index: Integer, index of the node
 |              from which to retrieve the attribute.
 |              E.g. `node_index=0` will correspond to the
 |              first input node of the layer.
```

```
|
|      Returns:
|          A tensor (or list of tensors if the layer has multiple inputs).
|
|      Raises:
|        RuntimeError: If called in Eager mode.
|
|  get_input_mask_at(self, node_index)
|      Retrieves the input mask tensor(s) of a layer at a given node.
|
|      Args:
|          node_index: Integer, index of the node
|              from which to retrieve the attribute.
|              E.g. `node_index=0` will correspond to the
|              first time the layer was called.
|
|      Returns:
|          A mask tensor
|          (or list of tensors if the layer has multiple inputs).
|
|  get_input_shape_at(self, node_index)
|      Retrieves the input shape(s) of a layer at a given node.
|
|      Args:
|          node_index: Integer, index of the node
|              from which to retrieve the attribute.
|              E.g. `node_index=0` will correspond to the
|              first time the layer was called.
|
|      Returns:
|          A shape tuple
|          (or list of shape tuples if the layer has multiple inputs).
|
|      Raises:
|        RuntimeError: If called in Eager mode.
|
|  get_output_at(self, node_index)
|      Retrieves the output tensor(s) of a layer at a given node.
|
|      Args:
|          node_index: Integer, index of the node
|              from which to retrieve the attribute.
|              E.g. `node_index=0` will correspond to the
|              first output node of the layer.
|
|      Returns:
|          A tensor (or list of tensors if the layer has multiple outputs).
|
|      Raises:
|        RuntimeError: If called in Eager mode.
|
|  get_output_mask_at(self, node_index)
|      Retrieves the output mask tensor(s) of a layer at a given node.
|
|      Args:
|          node_index: Integer, index of the node
|              from which to retrieve the attribute.
|              E.g. `node_index=0` will correspond to the
|              first time the layer was called.
|
|      Returns:
|          A mask tensor
|          (or list of tensors if the layer has multiple outputs).
|
|  get_output_shape_at(self, node_index)
|      Retrieves the output shape(s) of a layer at a given node.
|
|      Args:
|          node_index: Integer, index of the node
|              from which to retrieve the attribute.
|              E.g. `node_index=0` will correspond to the
|              first time the layer was called.
|
|      Returns:
|          A shape tuple
|          (or list of shape tuples if the layer has multiple outputs).
|
|      Raises:
|        RuntimeError: If called in Eager mode.
|
|  get_weights(self)
|      Returns the current weights of the layer, as NumPy arrays.
|
|      The weights of a layer represent the state of the layer. This function
|      returns both trainable and non-trainable weight values associated with this
|      layer as a list of NumPy arrays, which can in turn be used to load state
```

```
|      into similarly parameterized layers.
|
|      For example, a `Dense` layer returns a list of two values: the kernel matrix
|      and the bias vector. These can be used to set the weights of another
|      `Dense` layer:
|
|      >>> layer_a = tf.keras.layers.Dense(1,
|      ...   kernel_initializer=tf.constant_initializer(1.))
|      >>> a_out = layer_a(tf.convert_to_tensor([[1., 2., 3.]]))
|      >>> layer_a.get_weights()
|      [array([[1.],
|             [1.],
|             [1.]], dtype=float32), array([0.], dtype=float32)]
|      >>> layer_b = tf.keras.layers.Dense(1,
|      ...   kernel_initializer=tf.constant_initializer(2.))
|      >>> b_out = layer_b(tf.convert_to_tensor([[10., 20., 30.]]))
|      >>> layer_b.get_weights()
|      [array([[2.],
|             [2.],
|             [2.]], dtype=float32), array([0.], dtype=float32)]
|      >>> layer_b.set_weights(layer_a.get_weights())
|      >>> layer_b.get_weights()
|      [array([[1.],
|             [1.],
|             [1.]], dtype=float32), array([0.], dtype=float32)]
|
|      Returns:
|          Weights values as a list of NumPy arrays.
|
|  set_weights(self, weights)
|      Sets the weights of the layer, from NumPy arrays.
|
|      The weights of a layer represent the state of the layer. This function
|      sets the weight values from numpy arrays. The weight values should be
|      passed in the order they are created by the layer. Note that the layer's
|      weights must be instantiated before calling this function, by calling
|      the layer.
|
|      For example, a `Dense` layer returns a list of two values: the kernel matrix
|      and the bias vector. These can be used to set the weights of another
|      `Dense` layer:
|
|      >>> layer_a = tf.keras.layers.Dense(1,
|      ...   kernel_initializer=tf.constant_initializer(1.))
|      >>> a_out = layer_a(tf.convert_to_tensor([[1., 2., 3.]]))
|      >>> layer_a.get_weights()
|      [array([[1.],
|             [1.],
|             [1.]], dtype=float32), array([0.], dtype=float32)]
|      >>> layer_b = tf.keras.layers.Dense(1,
|      ...   kernel_initializer=tf.constant_initializer(2.))
|      >>> b_out = layer_b(tf.convert_to_tensor([[10., 20., 30.]]))
|      >>> layer_b.get_weights()
|      [array([[2.],
|             [2.],
|             [2.]], dtype=float32), array([0.], dtype=float32)]
|      >>> layer_b.set_weights(layer_a.get_weights())
|      >>> layer_b.get_weights()
|      [array([[1.],
|             [1.],
|             [1.]], dtype=float32), array([0.], dtype=float32)]
|
|      Args:
|        weights: a list of NumPy arrays. The number
|          of arrays and their shape must match
|          number of the dimensions of the weights
|          of the layer (i.e. it should match the
|          output of `get_weights`).
|
|      Raises:
|        ValueError: If the provided weights list does not match the
|          layer's specifications.
|
|  ----------------------------------------------------------------------
|  Readonly properties inherited from keras.engine.base_layer.Layer:
|
|  compute_dtype
|      The dtype of the layer's computations.
|
|      This is equivalent to `Layer.dtype_policy.compute_dtype`. Unless
|      mixed precision is used, this is the same as `Layer.dtype`, the dtype of
|      the weights.
|
|      Layers automatically cast their inputs to the compute dtype, which causes
|      computations and the output to be in the compute dtype as well. This is done
|      by the base Layer class in `Layer.__call__`, so you do not have to insert
```

```
|      these casts if implementing your own layer.
|
|      Layers often perform certain internal computations in higher precision when
|      `compute_dtype` is float16 or bfloat16 for numeric stability. The output
|      will still typically be float16 or bfloat16 in such cases.
|
|      Returns:
|        The layer's compute dtype.
|
|  dtype
|      The dtype of the layer weights.
|
|      This is equivalent to `Layer.dtype_policy.variable_dtype`. Unless
|      mixed precision is used, this is the same as `Layer.compute_dtype`, the
|      dtype of the layer's computations.
|
|  dtype_policy
|      The dtype policy associated with this layer.
|
|      This is an instance of a `tf.keras.mixed_precision.Policy`.
|
|  dynamic
|      Whether the layer is dynamic (eager-only); set in the constructor.
|
|  inbound_nodes
|      Return Functional API nodes upstream of this layer.
|
|  input
|      Retrieves the input tensor(s) of a layer.
|
|      Only applicable if the layer has exactly one input,
|      i.e. if it is connected to one incoming layer.
|
|      Returns:
|          Input tensor or list of input tensors.
|
|      Raises:
|        RuntimeError: If called in Eager mode.
|        AttributeError: If no inbound nodes are found.
|
|  input_mask
|      Retrieves the input mask tensor(s) of a layer.
|
|      Only applicable if the layer has exactly one inbound node,
|      i.e. if it is connected to one incoming layer.
|
|      Returns:
|          Input mask tensor (potentially None) or list of input
|          mask tensors.
|
|      Raises:
|          AttributeError: if the layer is connected to
|          more than one incoming layers.
|
|  input_shape
|      Retrieves the input shape(s) of a layer.
|
|      Only applicable if the layer has exactly one input,
|      i.e. if it is connected to one incoming layer, or if all inputs
|      have the same shape.
|
|      Returns:
|          Input shape, as an integer shape tuple
|          (or list of shape tuples, one tuple per input tensor).
|
|      Raises:
|          AttributeError: if the layer has no defined input_shape.
|          RuntimeError: if called in Eager mode.
|
|  losses
|      List of losses added using the `add_loss()` API.
|
|      Variable regularization tensors are created when this property is accessed,
|      so it is eager safe: accessing `losses` under a `tf.GradientTape` will
|      propagate gradients back to the corresponding variables.
|
|      Examples:
|
|      >>> class MyLayer(tf.keras.layers.Layer):
|      ...   def call(self, inputs):
|      ...     self.add_loss(tf.abs(tf.reduce_mean(inputs)))
|      ...     return inputs
|      >>> l = MyLayer()
|      >>> l(np.ones((10, 1)))
|      >>> l.losses
|      [1.0]
```

```
|
|      >>> inputs = tf.keras.Input(shape=(10,))
|      >>> x = tf.keras.layers.Dense(10)(inputs)
|      >>> outputs = tf.keras.layers.Dense(1)(x)
|      >>> model = tf.keras.Model(inputs, outputs)
|      >>> # Activity regularization.
|      >>> len(model.losses)
|      0
|      >>> model.add_loss(tf.abs(tf.reduce_mean(x)))
|      >>> len(model.losses)
|      1
|
|      >>> inputs = tf.keras.Input(shape=(10,))
|      >>> d = tf.keras.layers.Dense(10, kernel_initializer='ones')
|      >>> x = d(inputs)
|      >>> outputs = tf.keras.layers.Dense(1)(x)
|      >>> model = tf.keras.Model(inputs, outputs)
|      >>> # Weight regularization.
|      >>> model.add_loss(lambda: tf.reduce_mean(d.kernel))
|      >>> model.losses
|      [<tf.Tensor: shape=(), dtype=float32, numpy=1.0>]
|
|      Returns:
|        A list of tensors.
|
|  metrics
|      List of metrics added using the `add_metric()` API.
|
|      Example:
|
|      >>> input = tf.keras.layers.Input(shape=(3,))
|      >>> d = tf.keras.layers.Dense(2)
|      >>> output = d(input)
|      >>> d.add_metric(tf.reduce_max(output), name='max')
|      >>> d.add_metric(tf.reduce_min(output), name='min')
|      >>> [m.name for m in d.metrics]
|      ['max', 'min']
|
|      Returns:
|        A list of `Metric` objects.
|
|  name
|      Name of the layer (string), set in the constructor.
|
|  non_trainable_variables
|      Sequence of non-trainable variables owned by this module and its submodules.
|
|      Note: this method uses reflection to find variables on the current instance
|      and submodules. For performance reasons you may wish to cache the result
|      of calling this method if you don't expect the return value to change.
|
|      Returns:
|        A sequence of variables for the current module (sorted by attribute
|        name) followed by variables from all submodules recursively (breadth
|        first).
|
|  non_trainable_weights
|      List of all non-trainable weights tracked by this layer.
|
|      Non-trainable weights are *not* updated during training. They are expected
|      to be updated manually in `call()`.
|
|      Returns:
|        A list of non-trainable variables.
|
|  outbound_nodes
|      Return Functional API nodes downstream of this layer.
|
|  output
|      Retrieves the output tensor(s) of a layer.
|
|      Only applicable if the layer has exactly one output,
|      i.e. if it is connected to one incoming layer.
|
|      Returns:
|        Output tensor or list of output tensors.
|
|      Raises:
|        AttributeError: if the layer is connected to more than one incoming
|          layers.
|        RuntimeError: if called in Eager mode.
|
|  output_mask
|      Retrieves the output mask tensor(s) of a layer.
|
|      Only applicable if the layer has exactly one inbound node,
```

```
|      i.e. if it is connected to one incoming layer.
|
|      Returns:
|          Output mask tensor (potentially None) or list of output
|          mask tensors.
|
|      Raises:
|          AttributeError: if the layer is connected to
|          more than one incoming layers.
|
|  output_shape
|      Retrieves the output shape(s) of a layer.
|
|      Only applicable if the layer has one output,
|      or if all outputs have the same shape.
|
|      Returns:
|          Output shape, as an integer shape tuple
|          (or list of shape tuples, one tuple per output tensor).
|
|      Raises:
|          AttributeError: if the layer has no defined output shape.
|          RuntimeError: if called in Eager mode.
|
|  trainable_variables
|      Sequence of trainable variables owned by this module and its submodules.
|
|      Note: this method uses reflection to find variables on the current instance
|      and submodules. For performance reasons you may wish to cache the result
|      of calling this method if you don't expect the return value to change.
|
|      Returns:
|        A sequence of variables for the current module (sorted by attribute
|        name) followed by variables from all submodules recursively (breadth
|        first).
|
|  trainable_weights
|      List of all trainable weights tracked by this layer.
|
|      Trainable weights are updated via gradient descent during training.
|
|      Returns:
|        A list of trainable variables.
|
|  updates
|
|  variable_dtype
|      Alias of `Layer.dtype`, the dtype of the weights.
|
|  variables
|      Returns the list of all layer variables/weights.
|
|      Alias of `self.weights`.
|
|      Note: This will not track the weights of nested `tf.Modules` that are not
|      themselves Keras layers.
|
|      Returns:
|        A list of variables.
|
|  weights
|      Returns the list of all layer variables/weights.
|
|      Returns:
|        A list of variables.
|
|  ----------------------------------------------------------------------
|  Data descriptors inherited from keras.engine.base_layer.Layer:
|
|  activity_regularizer
|      Optional regularizer function for the output of this layer.
|
|  input_spec
|      `InputSpec` instance(s) describing the input format for this layer.
|
|      When you create a layer subclass, you can set `self.input_spec` to enable
|      the layer to run input compatibility checks when it is called.
|      Consider a `Conv2D` layer: it can only be called on a single input tensor
|      of rank 4. As such, you can set, in `__init__()`:
|
|      ```python
|      self.input_spec = tf.keras.layers.InputSpec(ndim=4)
|      ```
|
|      Now, if you try to call the layer on an input that isn't rank 4
|      (for instance, an input of shape `(2,)`, it will raise a nicely-formatted
```

```
|      error:
|
|      ```
|      ValueError: Input 0 of layer conv2d is incompatible with the layer:
|      expected ndim=4, found ndim=1. Full shape received: [2]
|      ```
|
|      Input checks that can be specified via `input_spec` include:
|      - Structure (e.g. a single input, a list of 2 inputs, etc)
|      - Shape
|      - Rank (ndim)
|      - Dtype
|
|      For more information, see `tf.keras.layers.InputSpec`.
|
|      Returns:
|        A `tf.keras.layers.InputSpec` instance, or nested structure thereof.
|
|  stateful
|
|  supports_masking
|      Whether this layer supports computing a mask using `compute_mask`.
|
|  trainable
|
|  ----------------------------------------------------------------------
|  Class methods inherited from tensorflow.python.module.module.Module:
|
|  with_name_scope(method) from builtins.type
|      Decorator to automatically enter the module name scope.
|
|      >>> class MyModule(tf.Module):
|      ...   @tf.Module.with_name_scope
|      ...   def __call__(self, x):
|      ...     if not hasattr(self, 'w'):
|      ...       self.w = tf.Variable(tf.random.normal([x.shape[1], 3]))
|      ...     return tf.matmul(x, self.w)
|
|      Using the above module would produce `tf.Variable`s and `tf.Tensor`s whose
|      names included the module name:
|
|      >>> mod = MyModule()
|      >>> mod(tf.ones([1, 2]))
|      <tf.Tensor: shape=(1, 3), dtype=float32, numpy=..., dtype=float32)>
|      >>> mod.w
|      <tf.Variable 'my_module/Variable:0' shape=(2, 3) dtype=float32,
|      numpy=..., dtype=float32)>
|
|      Args:
|        method: The method to wrap.
|
|      Returns:
|        The original method wrapped such that it enters the module's name scope.
|
|  ----------------------------------------------------------------------
|  Readonly properties inherited from tensorflow.python.module.module.Module:
|
|  name_scope
|      Returns a `tf.name_scope` instance for this class.
|
|  submodules
|      Sequence of all sub-modules.
|
|      Submodules are modules which are properties of this module, or found as
|      properties of modules which are properties of this module (and so on).
|
|      >>> a = tf.Module()
|      >>> b = tf.Module()
|      >>> c = tf.Module()
|      >>> a.b = b
|      >>> b.c = c
|      >>> list(a.submodules) == [b, c]
|      True
|      >>> list(b.submodules) == [c]
|      True
|      >>> list(c.submodules) == []
|      True
|
|      Returns:
|        A sequence of all submodules.
|
|  ----------------------------------------------------------------------
|  Static methods inherited from keras.utils.version_utils.LayerVersionSelector:
|
```

```
 |  __new__(cls, *args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate signature.
```

```
weights = gru.get_weights()
```

```
len(weights)
```

```
3
```

```
weights[0].shape
```

```
(8, 12)
```

```
weights[1].shape
```

```
(4, 12)
```

```
weights[2].shape
```

```
(2, 12)
```

Note que los pesos de la capa GRU Están organizados de la siguiente forma

```
units = 4 # tamaño del estado oculto
W = gru.get_weights()[0]
W_z = W[:,:units]
W_r = W[:, units:units*2]
W_n = W[:, units*2:units*3]


U = gru.get_weights()[1]
U_z = W[:,:units]
U_r= W[:, units:units*2]
U_n = W[:, units*2:units*3]

b = gru.get_weights()[2]
b_z = b[:units]
b_r = b[ units:units*2]
b_n = b[units*2:units*3]
```

Referencias

1. Introducción a Redes LSTM
2. Time Series Forecasting with LSTMs using TensorFlow 2 and Keras in Python
3. Dive into Deep Learnig
4. Understanding LSTM Networks
5. Ralf C. Staudemeyer and Eric Rothstein Morris,*Understanding LSTM a tutorial into Long Short-Term Memory Recurrent Neural Networks*, arxiv, September 2019
6. Karpathy, *The Unreasonable Effectiveness of Recurrent Neural Networks*
7. Cho et al. 2014 On the Properties of Neural Machine Translation: Encoder–Decoder Approaches
8. J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling

By Álvaro Mauricio Montenegro Díaz, Daniel Mauricio Montenegro Reyes
© Copyright 2022.